

Your name: Shreeya S. Kangutkar

Your Student ID: 40481802

READ ME: You must submit your assignment using this template. Do not amend the length or structure of this document, or the typeface or its size. Tasks stated here are simplified to keep the template concise. Comprehensive guidance for this assignment is available on the ELE8094 Canvas page. **YOU CAN DELETE THIS README SECTION ONCE READ.**

A. A one-page summary of your system’s functionality, i.e., what analysis your system performs to provide insights into the security threat landscape.

System Overview. Pure Secure is a three-tier web application implementing the User–System–CWE architecture required in the brief. The frontend (HTML/CSS/JavaScript, served on localhost:8080) communicates with a Python FastAPI backend, which ingests the MITRE CWE XML dataset and proxies real-time vulnerability data from the NIST National Vulnerability Database (NVD). Authentication forms the trust boundary: any request lacking a valid JSON Web Token (JWT) is rejected before data is loaded or rendered, ensuring that all CWE and CVE information is accessible only to authenticated users.

Core functionality. Once authenticated, users access five functional areas. The **Home** page provides a quick-search interface into the CWE dataset. The **Search** page performs full-text queries by CWE ID, name, or description—for example, “CWE-778” returns *Insufficient Logging* with severity and description. The **Browse** page loads the full dataset and supports filtering by severity tier (Critical, High, Medium, Low) and category keyword, enabling structured exploration of the weakness landscape. The **CVE** page integrates with the NVD API, allowing retrieval of individual vulnerabilities (e.g., CVE-2024-36971) with CVSS score, affected CPEs, publication dates, and advisory references. A service-health check confirms NVD reachability before queries are issued. The **Statistics** page aggregates dataset-wide metrics: 969 total CWEs, 81 Critical, 87 High, 801 Medium, across 15 categories.

Analysis beyond the CWE site. The brief requires the system to provide insights not directly available from the CWE website. Pure Secure delivers three such capabilities:

1. **Dataset-level severity analytics.** MITRE presents CWEs individually; it does not provide cross-dataset severity distributions. Pure Secure Statistics page aggregates severity tiers across the entire dataset, supporting high-level risk assessment.
2. **CWE–CVE correlation.** The CVE page links weakness classes to real-world exploitation. For example, CVE-2024-36971 (Linux kernel use-after-free, CVSS 7.8) is presented alongside its CWE classification and affected components. This supports threat-informed decision-making that the CWE site alone cannot provide.
3. **Risk-prioritised browsing.** The Browse page enables analysts to filter by severity, allowing a triage workflow: Critical weaknesses first, then High, then Medium. This prioritisation is absent from the CWE site’s alphabetical listing and directly supports secure-development practices such as risk-based remediation.

Together, these features transform raw CWE and CVE data into actionable intelligence for security practitioners, meeting the assignment requirement for meaningful insights into the security threat landscape.

B. An explanation and justification of a part of your design that is significant for secure software (one page)

Design focus: JWT stateless authentication and fail-secure access control.

The most security-significant design decision in Pure Secure is the adoption of **JWT-based stateless authentication** as the trust boundary between the user and all data-bearing components. This was chosen over server-side session management to reduce attack surface, eliminate shared session state, and align with the principle of **least privilege**: only requests presenting a valid, verifiable token may access CWE or CVE data. Stateless verification aligns with NIST SP 800-63B guidance on tokens whose claims can be validated without server-side session storage [1].

How the design works. On successful login, the backend issues a signed JWT containing the user's identity. The frontend stores the token in local Storage and attaches it to every API request as a Bearer token. The backend verifies the token's signature on each request, ensuring authentication without maintaining server-side state.

Fail-secure defaults. Access control is enforced at two points:

1. **On page load**, a DOM Content Loaded handler checks for a token before any data is fetched. If absent, the user is redirected to login. This ensures the safe state is *no access*, embodying the **failure-secure** principle [2].
2. **On any 401 responses**, a centralised fetch wrapper deletes the token and redirects to login. This prevents partial or degraded access states and ensures expired or invalid tokens cannot be reused.

This design reduces inconsistent authentication logic across the application, supporting **defence in depth** and **separation of concerns**.

Separation of concerns. Authentication logic is isolated in auth-common.js and the /auth/ backend routes, while data logic resides in script.js and the /cwe/ and /cve/ routes. This limits the blast radius of defects: a rendering bug cannot affect credential verification.

Trade-offs and justification. Storing JWTs in localStorage exposes them to theft via XSS (CWE-79). A production system would use HttpOnly, Secure, SameSite cookies to prevent client-side access. This prototype accepts the trade-off due to localhost constraints and the absence of an HTTPS deployment environment. The risk is explicitly documented, demonstrating awareness of residual vulnerabilities and aligning with secure-by-design principles.

C. An explanation and justification of a part of your implementation that is significant for secure software (one page)

Implementation focus: input validation and output encoding.

Two weakness classes—CWE-20 (Improper Input Validation) and CWE-79 (Cross-Site Scripting)—are among the most prevalent in real systems. Pure Secure addresses through explicit, verifiable implementation decisions.

Strict input validation (CWE-20). User-supplied CVE identifiers are normalised and validated against a strict allow-list regex:

```
const CVE_REGEX = /^CVE-\d{4}-\d{4,}\$/;
```

Only structurally valid identifiers (CVE-YYYY-NNNNN) are accepted; all others are rejected before any network request is issued. This implements **positive validation**, as recommended by OWASP's Input Validation Cheat Sheet [3]. The backend applies a second layer of validation using Pydantic field constraints, rejecting malformed inputs with HTTP 422. This two-layer approach demonstrates **defence in depth**: the frontend improves usability, while the backend enforces the security boundary.

A limitation remains: regex validation ensures structural correctness, not semantic correctness (e.g., whether the CVE exists). This is acceptable for a prototype and documented as such.

Output encoding (CWE-79). All untrusted content returned from the backend or NVD API is passed through an escape HTML() function before insertion into the DOM:

```
function escapeHTML(str) {  
  const div = document.createElement('div');  
  div.textContent = str;  
  return div.innerHTML;  
}
```

Assigning to text Content delegates encoding to the browser's HTML parser, avoiding brittle custom sanitisation logic. This prevents script injection and aligns with OWASP's XSS Prevention Cheat Sheet [5].

Centralised API client and timeout controls. All backend communication passes through a single fetch API () wrapper that:

- attaches the JWT
- enforces a 10-second timeout using Abort Controller
- handles 401 responses centrally

This prevents inconsistent authentication handling and ensures that slow or unresponsive external dependencies (e.g., NVD) do not degrade availability. The explicit NVD health-check endpoint provides transparency to the user, reducing silent failure modes.

Together, these implementation decisions demonstrate **secure coding**, **defence in depth**, and **robust error handling**, directly addressing high-impact CWE classes.

D. An explanation and justification of a part of your Dev SecOps process that is significant for secure software (one page).

Focus: version control, environment separation, and a security-focused CI/CD pipeline.

Version control and traceability. All source files—frontend, backend, configuration, and pipeline definitions—are tracked in GitLab. Commits were structured around discrete functional units (authentication, CWE ingestion, CVE integration, statistics), providing a clear audit trail of when security-relevant changes entered the codebase. This aligns with NIST SSDF PO.3, which emphasises traceability for vulnerability investigation and change control [4].

Environment separation. The `config.js` module automatically selects the correct backend URL based on `window.location.protocol`, ensuring that development builds use `localhost:8000` while production builds use the Render-hosted HTTPS endpoint. Centralising this logic prevents misconfiguration errors where a production frontend contacts a development backend with weaker controls. This supports the assurance principles of **configuration integrity** and **attack surface reduction**.

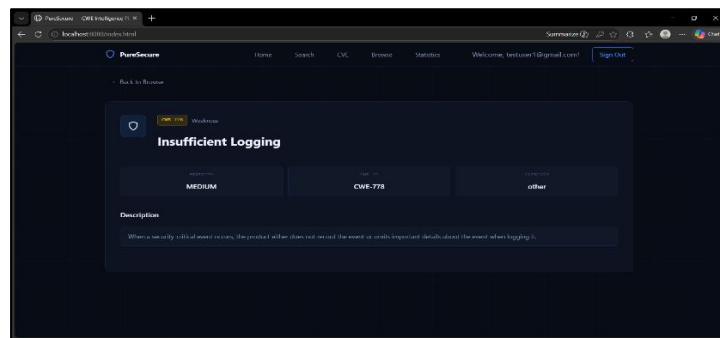
Shift-left security. Security controls were implemented alongside the features they protect: authentication checks were added during initial page-load logic, output encoding was implemented before any rendering, and CVE validation was introduced with the search feature. This operationalises NIST SSDF PO.5's requirement to integrate security early and continuously [4].

CI/CD pipeline with security gates. The `.gitlab-ci.yml` defines three sequential stages:

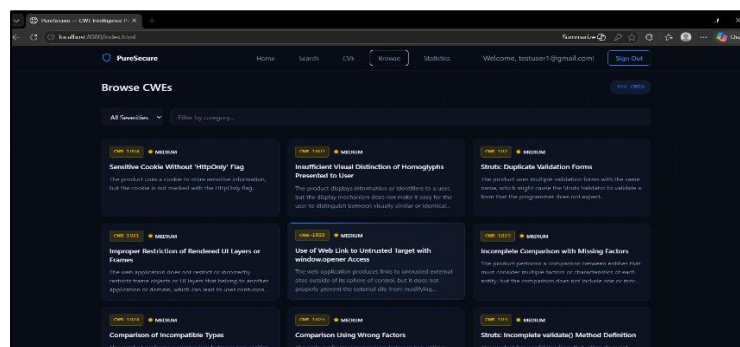
1. **test** — `pytest` verifies that protected endpoints (`/cwe/`, `/cve/search`, `/cwe/stats`) return 401 without a valid token. This directly tests security properties rather than inferring them from functional behaviour.
2. **security-scan** — GitLab SAST scans JavaScript and Python for unsafe patterns; Trivy checks Python dependencies (FastAPI, PyJWT, uvicorn) against the NVD CVE database. The pipeline fails on HIGH or CRITICAL findings, acting as an automated security gate.
3. **deploy** — a Render webhook triggers deployment only on merges to main. Branch protection requires at least one approved merge request, ensuring human review complements automated scanning.

Evaluation. This pipeline mitigates risks such as dependency vulnerabilities, insecure coding patterns, and unreviewed changes reaching production. Limitations remain: SAST may miss logic flaws, and dependency scanning cannot detect vulnerabilities in custom code. These limitations justify the combined use of automated tools and human review, aligning with secure-development best practices.

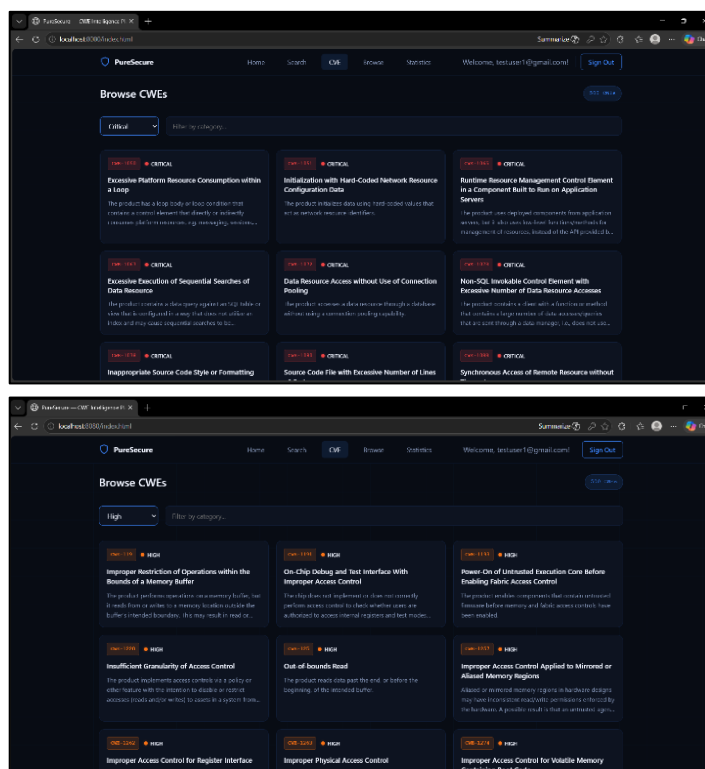
(4) CWE Detail view for CWE-778 showing Severity, ID, Category and Description fields.

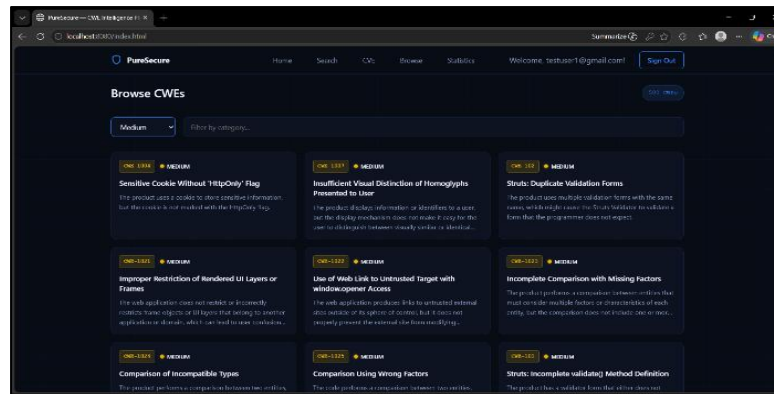


(5) Browse page filtered by All Severities.

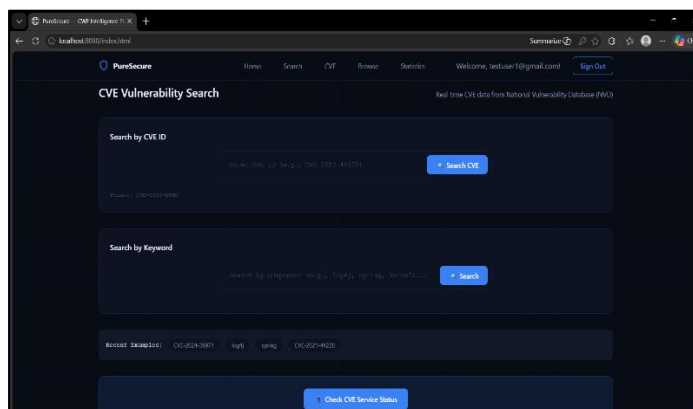


(6) Browse filtered by Critical, High, Medium (CWE-119, CWE-125, CWE-1220).

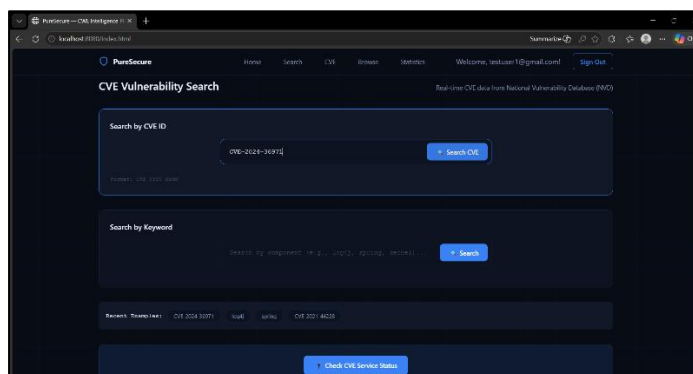




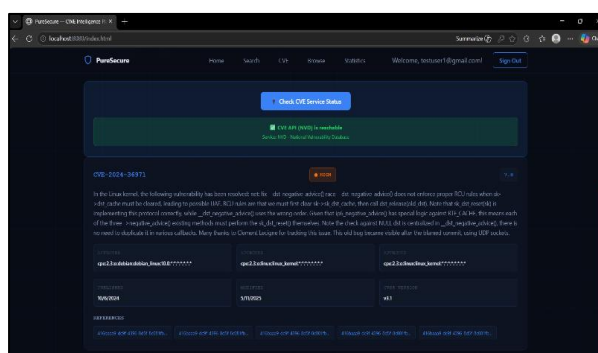
(7) CVE Vulnerability Search page with NVD integration and format hint.



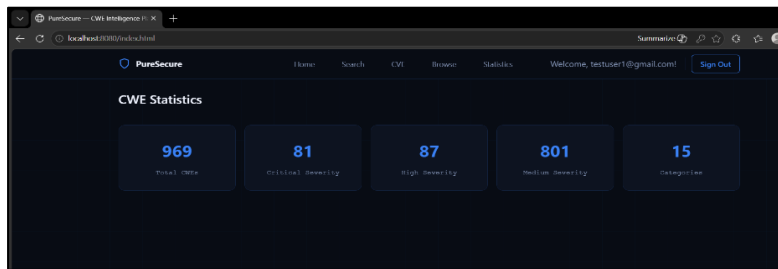
(8) CVE-2024-36971 entered in search field.



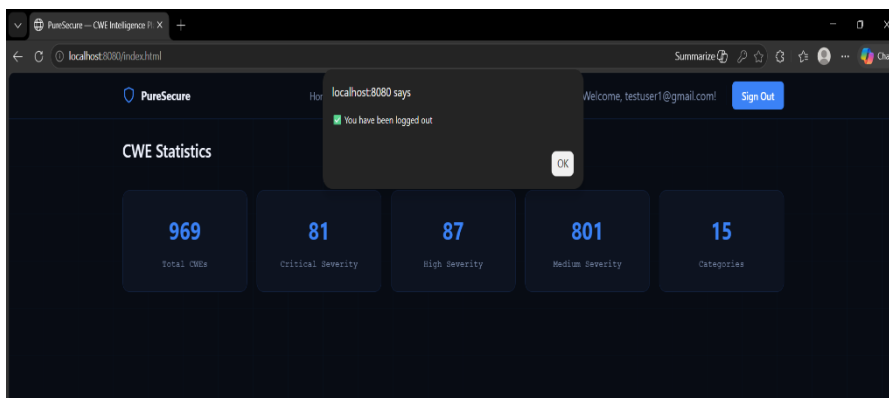
(9) CVE-2024-36971 result: HIGH severity



10) Static Page:



11) Logout confirmation dialog.



12) CWE Database

```

Selection View Go Run ... ← → puresecure
more  README.md  cwe_v4.19.1.xml X
1  >>Weakness_Catalog Name="CWE" Version="4.19.1" Date="2026-01-21" xmlns="http://cwe.mitre.org/cwe-7" xmlns:xsi="http://www.w3.org/2001/XM
2
3  tive Cookie Without 'HttpOnly' Flag" Abstraction="Variant" Structure="Simple" Status="Incomplete">
4  s a cookie to store sensitive information, but the cookie is not marked with the HttpOnly flag.</Description>
5  tpOnly flag directs compatible browsers to prevent client-side script from accessing cookies. Including the HttpOnly flag in the Set-Coo
6
7  "ChildOf" CWE_ID="732" View_ID="1000" Ordinal="Primary"/>
8

```

13) CVE Database

```

backend > app > cve > cve_service.py > ...
1  """
2  CVE Service - Real-time CVE data fetching from external APIs.
3  Handles API calls, response parsing, and error handling.
4  """
5
6  import logging
7  import requests
8  from typing import Dict, List, Optional
9  from fastapi import HTTPException, status
10 from app.cve.cve_schema import CVEResponse, CVEMetrics, CVEReference
11
12 logger = logging.getLogger(__name__)
13
14 # NVD API configuration
15 NVD_API_BASE = "https://services.nvd.nist.gov/rest/json/cves/2.0"
16 REQUEST_TIMEOUT = 5 # seconds
17 MAX_RESULTS = 20
18

```

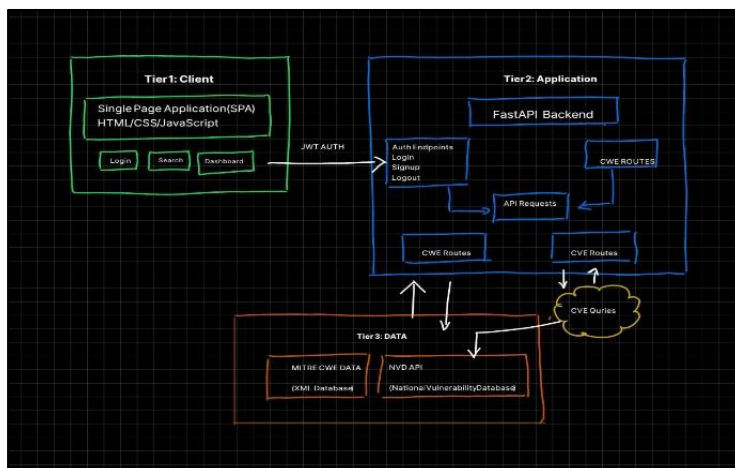

14) GitLab CI/CD pipeline

```

1 # .gitlab-ci.yml
2 stages:
3   - test
4   - security-scan
5   - deploy
6
7 variables:
8   PIP_CACHE_DIR: "${CI_PROJECT_DIR}/.cache/pip"
9
10 run-tests:
11   stage: test
12   image: python:3.11-slim
13   before_script:
14     - pip install fastapi uvicorn pyjwt pytest httpx --break-system-packages
15   script:
16     - pytest tests/ -v --tb=short
17
18 sast:
19   stage: security-scan
20   include:
21     - template: Security/SAST.gitlab-ci.yml
22
23 dependency-scan:
24   stage: security-scan
25   image: python:3.11-slim
26   script:
27     - pip install pip-audit --break-system-packages
28     - pip-audit --requirement requirements.txt --format table
29   allow_failure: true
30
31 deploy:
32   stage: deploy
33   script:
34     - echo "Deploy stage placeholder - no live environment configured."
35   only:
36     - main

```

b) An appendix containing your software designs, with appropriate explanations.



Pure Secure follows a three-tier architecture.

Tier 1 (Client):

HTML/CSS/JavaScript SPA comprising login.html, signup.html and index.html.

Navigation between pages (Home, Search, CVE, Browse, Statistics) is handled by client-side & navigate to script.js without full page reloads. All API calls are routed through the centralised fetch API () wrapper, which attaches the JWT Bearer token, enforces a 10-second timeout, and handles 401 responses.

Tier 2 (Application):

Python Fast API backend exposing versioned REST endpoints. Authentication routes (/auth/login, /auth/signup, /auth/logout) issue and validate JWTs. CWE routes serve normalised weakness data ingested from the MITRE XML dataset. CVE routes proxy requests to the NVD API and return structured results.

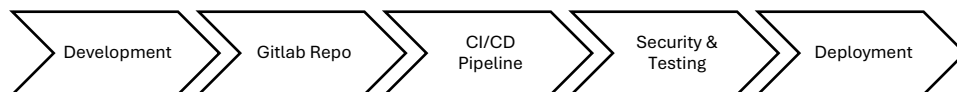
Tier 3 (Data):

MITRE CWE XML dataset (parsed and stored by the backend) and NIST NVD REST API (queried at request time). The trust boundary sits between Tier 1 and Tier 2: no CWE or CVE data is accessible without a valid JWT.

- c) A link to your GitLab project showing the work (e.g. code) you have done for the assignment.

[git@gitlab.eecs.qub.ac.uk:40481802/puresecure1.git](https://gitlab.eecs.qub.ac.uk/40481802/puresecure1.git)
<https://gitlab.eecs.qub.ac.uk/40481802/puresecure1.git>

- d) appendix containing a summary of your DevSecOps process.



Development was conducted locally using Git with a structured feature-branching workflow, while repositories were hosted and managed on GitLab. Commits were logically organised by functional domains, including authentication, CWE data handling, CVE integration, statistics, and configuration management, ensuring clear traceability and maintainability. The config.js module was designed to enforce environment separation between development and production contexts, enabling seamless transitions between a local development server (localhost:8080) and a deployed HTTPS endpoint on Render without requiring code modifications. Security considerations were integrated throughout the development lifecycle rather than applied retrospectively. Specifically, authentication validation using JSON Web Tokens was enforced on DOM Content Loaded, output sanitisation was consistently applied via an escape HTML () utility at all rendering points to mitigate injection risks, and strict input validation using a CVE-format regular expression was implemented at the search boundary.

The CI/CD pipeline has been designed and its configuration committed to the repository as .gitlab-ci.yml. The pipeline structure is as follows:

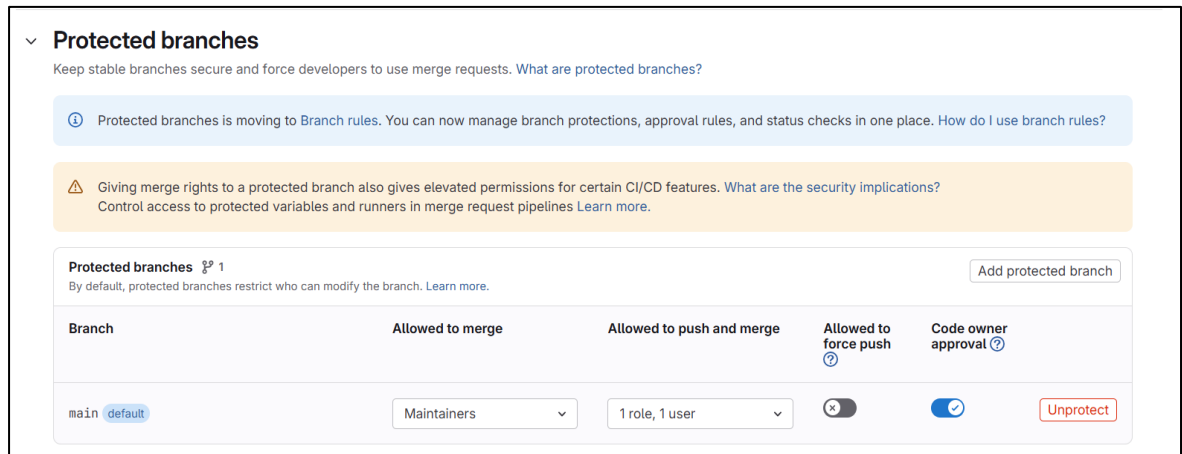
Stage 1 — test: pytest security assertions on authentication endpoints (HTTP 401 without Bearer token verified programmatically for /cwe/, /cve/search, /cwe/stats).

Stage 2 — security-scan: GitLab SAST template (JavaScript + Python); Trivy dependency scan on all Python packages against NVD CVE database; pipeline configured to block on HIGH/CRITICAL findings.

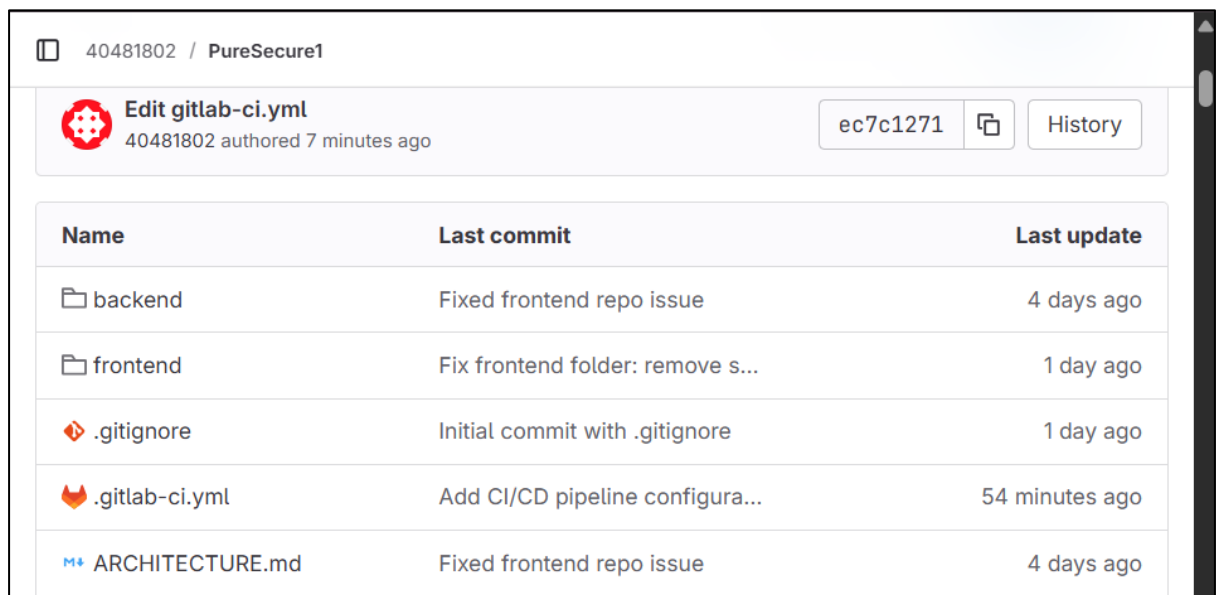
Stage 3 — deploy: Render webhook trigger on main branch only, guarded by branch protection requiring one approved merge request.

Branch protection: direct pushes to main are disabled; all changes require a merge request with one approver, ensuring human review complements automated scanning.

i) GitLab repository Settings → Protected Branches: main branch with direct pushes blocked and merge request approval required before merge.



ii) GitLab repository file tree showing `.gitlab-ci.yml` committed to the root of main, evidencing the pipeline configuration is version-controlled alongside application code.



e) An appendix containing references to sources (where appropriate)

[1] NIST, Digital Identity Guidelines, SP 800-63B, 2017. <https://pages.nist.gov/800-63-3/sp800-63b.html>

[2] G. McGraw, Software Security: Building Security In. Addison-Wesley, 2006.

[3] OWASP, Input Validation Cheat Sheet.

https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

[4] NIST, Secure Software Development Framework (SSDF), SP 800-218, 2022.

<https://csrc.nist.gov/publications/detail/sp/800-218/final>

[5] OWASP, XSS Prevention Cheat Sheet.

https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

[6] MITRE, Common Weakness Enumeration. <https://cwe.mitre.org/>

Documentation of the use of AI tools

README: Clearly document all uses you make of AI tools in your assignment. You should not use any output from AI directly in any of your answers, including in citations or references.